

Technical Description of the Project Architecture, Data Pipeline, and Training Workflow

(Updated for End-to-End Architecture)

ml2_final_project

May 30, 2026

Abstract

This document explains the project at the level a new reader needs first: how the repository is organized, how the data moves from raw corpora to processed tensors, how the training code is split into reusable modules, and how the experiment folders are produced. The architecture has been updated to an End-to-End (E2E) framework utilizing a Global Style Token (GST) and Normalizing Flows to allow real-time cross-lingual voice conversion.

Contents

1	Proposal	3
2	Repository Overview	3
2.1	High-Level Directory Tree	3
2.2	What Each Major Folder Means	4
3	End-to-End Data Flow	4
3.1	Data Pipeline Diagram	5
4	How Data Should Be Organized	5
4.1	Raw Data Layout	5
4.2	Processed Data Layout	5
4.3	What Goes Into a .pt File	6
4.4	Example Sample Payload	6
4.5	How the Pipeline Fits Together	6
4.6	Example Commands and How to Use Them	7
4.7	How to Read the Flags	7
5	System Architecture: Train Structure	8
5.1	Pipeline Diagram	8
5.2	Architecture Specifications and Dimensions	8
6	Loss Functions	9
7	Inference Workflow	10
7.1	Inference Architecture Diagram	10

8	Training Code Organization in src	10
8.1	Module Map	10
9	How to Run Experiments & Training Phases	10
9.1	Phase 1: Flow & Alignment (Epochs 0 - 100)	11
9.2	Phase 2: Acoustic Fine-Tuning (Epochs 101 - 250)	11
9.3	Phase 3: End-to-End Joint Training (Epochs 251 - 500)	11
9.4	Example Commands	11
10	Summary	12

1 Proposal

The goal of this project is to build an end-to-end, real-time model for cross-lingual voice conversion. The model will be trained with the following datasets: LibriSpeech-PT, TTS Portuguese Corpus, and LibriSpeech-EN. It leverages pre-trained foundational models (XLM-RoBERTa for text and HuBERT for audio) combined with a Global Style Token (GST) to disentangle language content from speaker prosody.

2 Repository Overview

The repository is divided into a few clear responsibilities: raw data storage, preprocessing, reusable code, experiment outputs, and documentation.

2.1 High-Level Directory Tree

```
ml2_final_project/
|-- data/
|   |-- raw/
|   |   |-- libriSpeech-pt/
|   |   '--- tts-portuguese-Corpora/
|   |-- processed/
|   |   |-- libriSpeech-pt/
|   |   |   |-- train/
|   |   |   |   '--- librispeech_mels_metadata.csv
|   |   |   '--- test/
|   |   |       '--- librispeech_mels_metadata.csv
|   |   '--- tts_portuguese/
|   |       '--- mels_metadata.csv
|   |-- interim/
|   '--- external/
|-- scripts/
|   |-- download_datasets/
|   '--- preprocess/
|       |-- preprocess_libriSpeech-pt.py
|       '--- preprocess_mels_tts_portuguese.py
|-- src/
|   |-- data/
|   |   '--- first_step_data_loaders/
|   |       '--- datasets.py
|   |-- models/
|   |   |-- PosteriorEncoder.py
|   |   |-- GST.py
|   |   |-- TextEncoder.py
|   |   |-- NormalizingFlow.py
|   |   '--- HiFi-GAN.py
|   '--- training/
|       '--- train_first_step/
|           |-- train.py
|           |-- model_loader.py
|           |-- losses.py
|           |-- train_utils.py
|           '--- text_processing.py
```

```
|
|         |-- configs.py
|         |-- run_training.py
|         '-- checkpoint_utils.py
|-- experiments/
|   '-- step_1/
|       '-- attempt_<timestamp>/
'-- docs/
```

2.2 What Each Major Folder Means

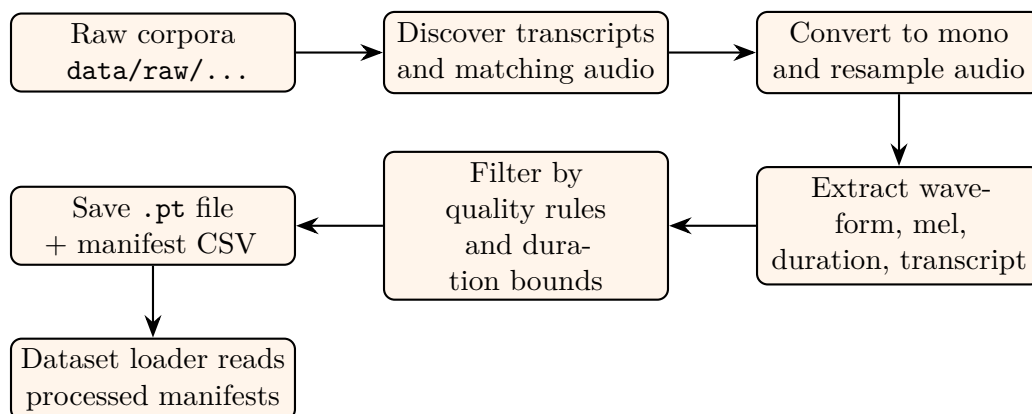
Folder	Role in the project
<code>data/raw</code>	Original corpora exactly as downloaded. Nothing here should depend on a model version or a feature extraction recipe.
<code>data/processed</code>	Processed artifacts such as <code>.pt</code> files and metadata CSV manifests used by the dataset layer.
<code>data/interim</code>	Temporary intermediate outputs or scratch files.
<code>scripts/preprocess</code>	Reproducible preprocessing scripts that convert raw audio into filtered, normalized, indexed training samples.
<code>src/data</code>	Dataset wrappers and batching helpers that turn manifests into PyTorch objects.
<code>src/models</code>	Reusable neural network components (GST, Encoders, Flows, Vocoder).
<code>src/training</code>	Training orchestration, losses, configuration presets, checkpointing, and experiment utilities.
<code>experiments</code>	Timestamped or named experiment outputs. Each run keeps its config, checkpoints, logs, and TensorBoard data together.
<code>docs</code>	Technical descriptions, reports, and the LaTeX sources.

3 End-to-End Data Flow

The project follows a simple lifecycle:

1. Download raw corpora into `data/raw`.
2. Preprocess audio and transcripts into standardized tensors.
3. Save per-utterance artifacts and metadata manifests in `data/processed`.
4. Load the processed manifests with the dataset classes in `src/data`.
5. Build the training experiment from the modules in `src/training`.
6. Save checkpoints, logs, and plots under `experiments/step_1`.

3.1 Data Pipeline Diagram



4 How Data Should Be Organized

4.1 Raw Data Layout

Place each corpus in its own directory under `data/raw`. A practical example is:

```
data/raw/  
|-- libriSpeech-pt/  
|   '-- mls_portuguese_opus/  
|       |-- train/  
|       |-- dev/  
|       '-- test/  
'-- tts-portuguese-Corpora/  
    '-- TTS-Portuguese-Corpus/  
        |-- audio/  
        |-- text/  
        '-- metadata/
```

In this structure, the preprocessing scripts can discover transcripts and audio without mixing source corpora or overwriting the downloaded archives.

4.2 Processed Data Layout

The processed area mirrors the dataset identity and the split. The dataset classes use manifests directly. A representative layout is:

```
data/processed/  
|-- libriSpeech-pt/  
|   |-- train/  
|   |   |-- pt_000123.pt  
|   |   |-- pt_000124.pt  
|   |   '-- librispeech_mels_metadata.csv  
|   '-- test/  
|       |-- pt_010001.pt  
|       '-- librispeech_mels_metadata.csv  
'-- tts_portuguese/  
    |-- ttsp_000045.pt  
    '-- mels_metadata.csv
```

Example manifest row:

```
utt_id,text,duration,mel_path,audio_path,source
pt_000123,ola mundo,2.83,data/processed/libriSpeech-pt/train/pt_000123.pt,data/raw/
libriSpeech-pt/.../pt_000123.flac,libriSpeech-pt
```

4.3 What Goes Into a .pt File

Each serialized sample should keep everything needed to train:

- **waveform**: the resampled mono waveform tensor (22050Hz).
- **mel**: the log-mel spectrogram tensor, with 80 mel bins.
- **sr**: the sample rate used by preprocessing.
- **duration**: utterance duration in seconds.
- **text**: the transcript text after normalization.
- **utt_id**: the unique utterance identifier.

4.4 Example Sample Payload

One saved example may contain the following keys and values:

```
{
  "waveform": Tensor([1, T]),
  "mel": Tensor([80, M]),
  "sr": 22050,
  "duration": 2.83,
  "text": "ola mundo",
  "utt_id": "pt_000123",
  "audio_path": "data/raw/libriSpeech-pt/.../pt_000123.flac"
}
```

Note on Mel-Spectrograms: Since the original ‘pt’ files contain mels that are not natively in the optimal latent space for HiFi-GAN, a *Posterior Encoder* is introduced in the architecture to project these 80-bin mels into the correct latent space z .

4.5 How the Pipeline Fits Together

The full pipeline works as a sequence of reusable steps:

1. Raw corpora are downloaded into **data/raw**.
2. The preprocessing scripts discover audio files and transcripts.
3. Audio is normalized, resampled, and converted into mel tensors.
4. Each utterance is stored as one **.pt** file plus a manifest row.
5. The dataset loaders read the manifests and reconstruct training examples.
6. Training code consumes the examples and writes experiment outputs under **experiments/step_1**.

4.6 Example Commands and How to Use Them

Run these commands from the repository root. The backslash at the end of a line means the command continues on the next line, which keeps long commands readable.

```
# Preprocess LibriSpeech-PT into processed tensors and a manifest
python scripts/preprocess/preprocess_libriSpeech-pt.py \
  --input-dir data/raw/libriSpeech-pt/mls_portuguese_opus \
  --out-dir data/processed/libriSpeech-pt/train \
  --target-sr 22050 \
  --n-mels 80 \
  --n-fft 1024 \
  --hop-length 256 \
  --win-length 1024 \
  --min-duration 1.0 \
  --max-duration 10.0

# Preprocess the TTS Portuguese corpus
python scripts/preprocess/preprocess_mels_tts_portuguese.py \
  --input-dir data/raw/tts-portuguese-Corpora/TTS-Portuguese-Corpus \
  --out-dir data/processed/tts_portuguese \
  --target-sr 22050

# Start a quick training smoke test
python src/training/train_first_step/run_training.py quick_test

# Start a larger preset
python src/training/train_first_step/run_training.py balanced

# Launch a custom training run
python src/training/train_first_step/train.py \
  --num-epochs 100 \
  --batch-size 32 \
  --learning-rate 1e-3 \
  --weight-reconstruction 1.0 \
  --weight-diversity 0.5 \
  --num-workers 4 \
  --experiment-name cross_lingual_training
```

Use the preprocessing commands first so the processed directories exist before training starts. Use the preset commands when you want a reproducible configuration quickly, and use the custom command when you need to change the batch size, learning rate, or loss weights.

4.7 How to Read the Flags

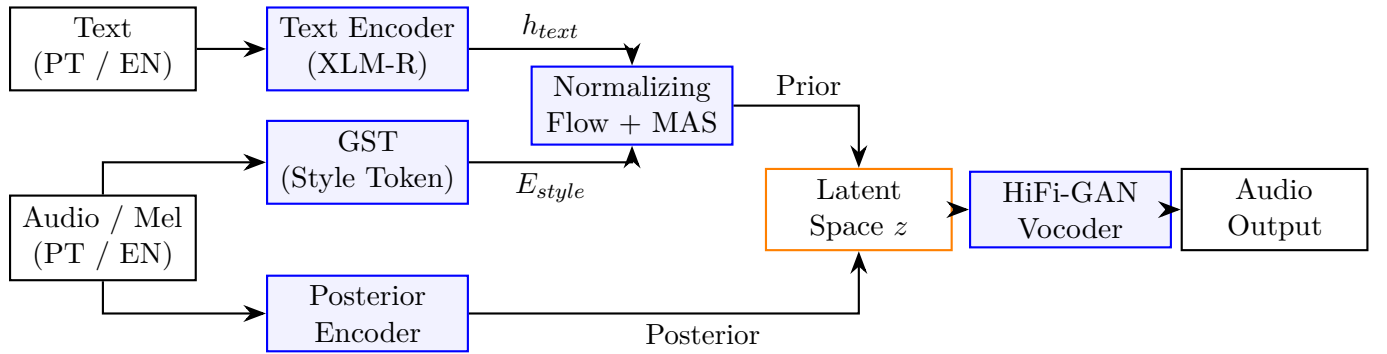
- `-input-dir`: points to the raw corpus folder.
- `-out-dir`: tells the script where to write processed files.
- `-target-sr`: sets the sample rate used during preprocessing.
- `-num-epochs`: controls how many epochs the trainer will run.
- `-batch-size`: sets the number of samples per optimization step.

- `-learning-rate`: sets the optimizer step size.
- `-weight-reconstruction` and `-weight-diversity`: control how the loss terms are balanced.
- `-experiment-name`: names the run directory under `experiments/step_1`.

5 System Architecture: Train Structure

The updated architecture replaces the cascaded Cross-Attention approach with an End-to-End (E2E) structure based on VITS principles. It utilizes a Posterior Encoder to map the training Mel-spectrograms into a unified latent space z , which the HiFi-GAN Vocoder is optimized to decode. Simultaneously, a Global Style Token (GST) extracts prosody and injects it into a Normalizing Flow, mapping the Text representations to the same latent space z .

5.1 Pipeline Diagram



During training, the **Posterior Encoder** turns the processed 80-bin mel tensors into latent space z . The **GST** computes a global style embedding. The **Normalizing Flow** aligns text with audio frames. The vocoder is trained on the extracted z .

5.2 Architecture Specifications and Dimensions

To successfully implement this E2E architecture, the modules must adhere to specific tensor dimensions and layer configurations.

- **Text Encoder (XLM-RoBERTa Base):**
 - **Input:** Tokenized text sequence.
 - **Architecture:** 12 Transformer layers, 12 attention heads.
 - **Output (h_{text}):** Sequence of embeddings with dimension $\mathbb{R}^{T_{text} \times 768}$.
- **Global Style Token (GST):**
 - **Input:** 80-bin Mel-spectrogram of the reference audio.
 - **Architecture:** A 2D Convolutional reference encoder (6 layers) followed by Multi-Head Attention over a bank of learnable style tokens.
 - **Output (E_{style}):** A fixed-length global style embedding with dimension $\mathbb{R}^{256}$.

- **Posterior Encoder:**

- **Input:** 80-bin target Mel-spectrogram.
- **Architecture:** 16 layers of non-causal WaveNet residual blocks with dilation factors (e.g., 1, 2, 4, 8...).
- **Output (z):** Latent acoustic representation with dimension $\mathbb{R}^{T_{audio} \times 192}$.

- **Normalizing Flow:**

- **Input:** Projected text embeddings aligned via MAS, conditioned globally on E_{style} .
- **Architecture:** 4 to 6 Affine Coupling Layers. Each coupling layer uses 4 WaveNet residual blocks with hidden dimension 192.
- **Output:** Maps the prior distribution to the posterior latent space $\mathbb{R}^{T_{audio} \times 192}$.

- **HiFi-GAN Vocoder:**

- **Input:** Latent vector $z \in \mathbb{R}^{T_{audio} \times 192}$ (projected internally from 192 to the initial upsampling dimension, e.g., 512).
- **Architecture:** 4 Transposed Convolutions for upsampling. Suggested upsampling rates $k_u = [8, 8, 2, 2]$ for 22050 Hz. Multi-Receptive Field Fusion (MRF) modules added in parallel.
- **Output:** High-fidelity 1D waveform.

6 Loss Functions

To ensure high-fidelity audio, prosody transfer, and stable training across languages, the loss objective incorporates Adversarial Learning, Contrastive Learning, and Variational bounds. The complete objective is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{Recon}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}} + \lambda_{\text{InfoNCE}} \mathcal{L}_{\text{InfoNCE}} + \mathcal{L}_{\text{Adv}}(G) + \mathcal{L}_{\text{FM}}$$

Mathematical Specifications:

1. **Mel Reconstruction Loss ($\mathcal{L}_{\text{Recon}}$):** Stabilizes the generator early on. Computed between the ground truth mel and the mel extracted from the HiFi-GAN output.

$$\mathcal{L}_{\text{Recon}} = \|\text{Mel}_{\text{target}} - \text{Mel}(G(z))\|_1$$

2. **Kullback-Leibler Divergence ($\mathcal{L}_{\text{KL}}$):** Forces the Normalizing Flow’s prior distribution $p(z|c_{\text{text}}, E_{\text{style}})$ to match the Posterior Encoder’s output $q(z|\text{Mel}_{\text{target}})$.

$$\mathcal{L}_{\text{KL}} = \int q(z|\text{Mel}) \log \frac{q(z|\text{Mel})}{p(z|c_{\text{text}}, E_{\text{style}})} dz$$

3. **Contrastive Style Loss ($\mathcal{L}_{\text{InfoNCE}}$):** Enforces prosody and emotional consistency. It maximizes the cosine similarity between the GST token of the input audio (s_i) and the GST token extracted from the generated audio ($\hat{s}_i$), while pushing away negative samples in the batch.

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\text{sim}(s_i, \hat{s}_i)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(s_i, s_j)/\tau)}$$

4. **Adversarial & Feature Matching Loss ($\mathcal{L}_{\text{Adv}}, \mathcal{L}_{\text{FM}}$):** Standard HiFi-GAN Least-Squares losses evaluated via Multi-Period and Multi-Scale Discriminators (MPD/MSD).

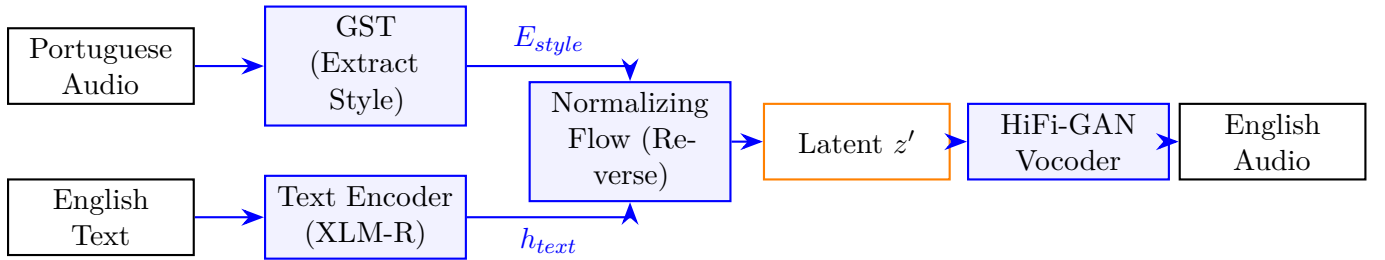
$$\mathcal{L}_{\text{Adv}}(G) = \mathbb{E}_z \left[(D(G(z)) - 1)^2 \right]$$

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{x,z} \left[\sum_{l=1}^L \frac{1}{N_l} \|D^{(l)}(x) - D^{(l)}(G(z))\|_1 \right]$$

7 Inference Workflow

During inference, we bypass the Posterior Encoder entirely. We extract the Style Token from the source Portuguese audio via the GST. We encode the English text and combine it with the Style Token. The Normalizing Flow runs in reverse to generate the latent variable z' . The Vocoder synthesizes the final English audio.

7.1 Inference Architecture Diagram



8 Training Code Organization in src

The training pipeline is split into small modules so that each concern has one main place in the codebase.

8.1 Module Map

Module	What it does
<code>train.py</code>	Main training entry point. Builds datasets and runs training.
<code>model_loader.py</code>	Loads the pre-trained components and assembles the network.
<code>losses.py</code>	Implements KL, InfoNCE, and adversarial losses.
<code>train_utils.py</code>	Contains epoch loops, checkpoints, and alignment helpers.

9 How to Run Experiments & Training Phases

Because the architecture utilizes massive pre-trained models, training must be phased to prevent catastrophic forgetting.

9.1 Phase 1: Flow & Alignment (Epochs 0 - 100)

- **Action:** Freeze XLM-R and HiFi-GAN Vocoder. Train only the Posterior Encoder, GST, and Normalizing Flows.
- **Goal:** Find the Monotonic Alignment (MAS) and optimize $\mathcal{L}_{KL}$.
- **Learning Rate:** 2×10^{-4} (AdamW).

9.2 Phase 2: Acoustic Fine-Tuning (Epochs 101 - 250)

- **Action:** Unfreeze HiFi-GAN generators and discriminators.
- **Goal:** Optimize $\mathcal{L}_{Adv}$, $\mathcal{L}_{FM}$, and $\mathcal{L}_{Recon}$ for high-fidelity audio synthesis. Incorporate InfoNCE for prosody.
- **Learning Rate:** 1×10^{-4} .

9.3 Phase 3: End-to-End Joint Training (Epochs 251 - 500)

- **Action:** Unfreeze the last 4 layers of XLM-R. Train the entire pipeline end-to-end.
- **Goal:** Minimize $\mathcal{L}_{total}$.
- **Learning Rate:** Drop foundational model LR to 1×10^{-5} , keep Flow/Vocoder at 1×10^{-4} .

9.4 Example Commands

```
# Phase 1: Warmup & Flow Alignment
python src/training/train_first_step/train.py \
  --num-epochs 100 \
  --batch-size 32 \
  --learning-rate 2e-4 \
  --freeze-text-encoder \
  --freeze-vocoder \
  --experiment-name phase1_flow_alignment

# Phase 3: E2E Joint Training
python src/training/train_first_step/train.py \
  --resume-from checkpoints/phase2_latest.pt \
  --num-epochs 250 \
  --batch-size 16 \
  --learning-rate-base 1e-5 \
  --learning-rate-flow 1e-4 \
  --lambda-kl 1.0 \
  --lambda-infonce 0.1 \
  --experiment-name phase3_e2e_finetune
```

10 Summary

The E2E architecture fundamentally resolves the bottleneck of translating Mel-spectrograms directly. By treating both languages and prosody as separate embeddings in a unified Normalizing Flow, the model achieves real-time inference with a single forward pass, heavily capitalizing on foundational weights (XLM-R) and high-fidelity vocoders (HiFi-GAN).